HOTELPRO HOSPITALITY MANAGEMENT PLATFORM - TECHNICAL REPORT

# HotelPro: Complete Hotel Booking and Management System

Comprehensive Technical Architecture, Database Design, System Security, and Stripe EUR Payment Workflow Specification

| | |
|---|---|
| **Document Version:** | 2.5.0 (Submission-Ready Architecture Specification) |
| **Project Codebase:** | HotelPro-Capstone (Full-Stack React + Express + MongoDB) |
| **Author and Role:** | Senior Software Architect, Technical Writer and Full-Stack Developer |
| **Target Evaluation:** | Capstone Defense, Production Architecture Audit and Submission |
| **Verification Status:** | 66 Automated Tests Passed \| 100% Code-Verified Implementation |
| **Publication Date:** | October 2026 |

*Notice: This technical documentation reflects the active, verified implementation of the HotelPro-Capstone repository. It contains no unverified claims or simulated outcomes.*

# *Executive Summary*

HotelPro is an enterprise-grade, full-stack hospitality management platform designed for luxury resorts, boutique heritage properties, and modern hotel chains. The platform delivers an end-to-end guest journey - from date-range suite discovery, real-time conflict-free reservation holds, and international currency conversions, to verified payment fulfillment, encrypted digital keycard issuance, and automated housekeeping turnovers.

Engineered on a modern decoupled architecture, HotelPro couples a React 19 single-page application built with Vite 8 and custom luxury Vanilla CSS with an Express 5 and Node.js 24 backend. Concurrency is deterministically managed through in-process Promise mutex locking per room ID, while online checkouts are protected by automated 15-minute hold expirations. International credit card transactions are settled seamlessly in Euros (EUR) via Stripe, with exact euro cent conversion and line-item tax invoice reconciliation.

# Chapter 1: Introduction

## 1.1 Project Title and Identity

Project Title: HotelPro - Complete Hotel Booking and Management System (Enterprise Hospitality Platform).

## 1.2 Abstract

In the modern luxury hospitality industry, guest satisfaction depends on seamless digital interactions, transparent cross-border billing, and prompt concierge service. Simultaneously, hotel operations require real-time synchronization between front desk clerks, housekeeping staff, and general managers. Traditional monolithic hotel systems frequently suffer from concurrency bottlenecks, race conditions during peak reservation windows, client-side pricing vulnerabilities, and rigid currency models that create discrepancies between quoted tariffs and final credit card charges.

HotelPro resolves these challenges through a modular, decoupled full-stack architecture. The platform combines a responsive React single-page frontend with an Express and MongoDB backend. Core capabilities include server-enforced tariff calculations (ignoring client-side overrides), in-process mutex locking to serialize room reservations, automated 15-minute reservation hold timeouts, international payment processing via Stripe in Euros (EUR) with mathematical currency conversion, digital tax invoice reconciliation, operational staff dashboards with digital keycard generation, and executive administrative KPI analytics.

## 1.3 Problem Statement

Hospitality management software faces several critical technical and operational challenges:

- Concurrent Overlapping Reservations: Multiple guests attempting to book the same luxury suite simultaneously risk double-booking race conditions without strict concurrency serialization.
- Client-Side Pricing Tampering: Systems that trust client-submitted prices are vulnerable to malicious HTTP payloads altering room rates.
- Cross-Border Currency Mismatch: International guests booking heritage properties quoted in domestic currencies (INR) require transparent, exact settlement in international currencies (EUR) without invoice rounding discrepancies.

- Uncontrolled Inventory Holds: Abandoned checkouts indefinitely lock room inventory unless an automated, time-bound expiration engine releases the dates.

- Operational Silos: Front desk check-ins, guest service tickets, and housekeeping sanitation statuses must be integrated into a unified, real-time platform.

## 1.4 Project Objectives

The HotelPro system was engineered to fulfill the following rigorous objectives:

- Enforce deterministic concurrency control via an in-process Promise mutex lock per room ID to eliminate double-booking race conditions.

- Calculate nights, tariffs, 12% statutory luxury taxes, and totals strictly on the server from authoritative database records.

- Implement an automated 15-minute hold expiration engine (holdExpiresAt) that frees abandoned rooms while permitting the same user to retry seamlessly.

- Deliver a verified Stripe EUR checkout pipeline converting canonical INR tariffs at a configured rate of 1 EUR = 110 INR down to smallest euro cents.

- Provide an alternative Pay at Hotel Desk workflow holding suites without expiration for in-person collection.

- Enforce strict Role-Based Access Control (RBAC) across Guests, Staff, and Administrators at both route middleware and database layers.

## 1.5 Functional and Non-Functional Requirements Summary

| Req ID | Category | Requirement Statement | Implementation Mechanism |
|---|---|---|---|
| FR-01 | Authentication | User registration and login issuing 7-day signed JWT tokens | bcryptjs 10 salt rounds + jsonwebtoken signed with JWT_SECRET |
| FR-02 | Authorization | Public registration forced to Guest role; privilege escalation blocked | Server hardcodes role: Guest in authRoutes.js |
| FR-03 | Inventory Discovery | Search and filter luxury suites by city, capacity, price, and amenities | GET /api/rooms with dynamic MongoDB query construction |
| FR-04 | Availability Filter | Exclude rooms with active confirmed bookings or unexpired holds | Half-open interval overlap math: checkIn < end and checkOut > start |
| FR-05 | Tariff Enforcement | Compute nights and 12% tax strictly from database rates | Room.findById authoritative pricing; client rates discarded |
| FR-06 | 15-Min Hold Expiry | Establish temporary 15-minute hold on suite during online checkout | holdExpiresAt timestamp verified by isHoldActive() helper |
| FR-07 | Stripe EUR Checkout | Convert INR tariff to EUR (1 EUR = 110 INR) and euro cents for Stripe | convertInrToEur() utility calling Stripe Sessions API |
| FR-08 | Pay at Hotel Desk | Allow in-person settlement with permanent hold and pending status | holdExpiresAt: null, paymentStatus: Pending, confirmed |
| FR-09 | Payment Verification | Dual-path verification: Stripe webhooks + client callback polling | constructEvent HMAC verification + POST /api/payments/stripe/verify |
| FR-10 | Digital Tax Invoice | Generate printable invoice reconciling line items and 12% tax | GET /api/payments/invoice/:id with roundingAdjustment |
| FR-11 | Digital Keycard | Staff check-in generating encrypted keycard and setting room Occupied | POST /api/staff/checkin creating CheckInOut document |
| FR-12 | Turnover Queuing | Staff check-out setting room Cleaning and queuing urgent service ticket | POST /api/staff/checkout creating ServiceRequest ticket |

| Req ID | Category | Requirement Statement | Implementation Mechanism |
|---|---|---|---|
| NFR-01 | Security | OWASP headers, nosniff, DENY frame options, CORS whitelisting | Server middleware blocking unauthorized domains and attacks |
| NFR-02 | Concurrency | Serialized execution of overlapping bookings on the same room | withRoomLock() Promise mutex keyed by room ID |

# Chapter 2: Technology Stack and Tools

HotelPro is built using a modern, verified JavaScript/Node.js technology stack across both frontend and backend layers. Version numbers reflect the actual dependencies specified in package files.

| Technology | Category | Verified Version | Purpose in HotelPro | Connection / Interaction Point |
|---|---|---|---|---|
| React | Frontend Core | 19.3.1 | Declarative UI rendering, state hooks, modal management | Renders SPA components in FrontEnd/src/ |
| Vite | Build Tooling | 8.3.4 | Next-gen development server, HMR, production bundling | Compiles frontend assets with React plugin |
| React Router DOM | Client Routing | 7.18.4 | Declarative client-side routing, protected route guards | Manages /rooms, /my-bookings, /staff, /admin |
| Axios | HTTP Client | 1.20.0 | REST API communication with Bearer token injection | Centralized axiosClient.js with request interceptors |
| Vanilla CSS | Styling System | Custom (20KB) | Bespoke luxury design language, gold/ivory color tokens | index.css containing glassmorphism and keyframes |
| Node.js | Backend Runtime | 24.14.0 | Asynchronous event-driven server execution runtime | Executes server.js and backend utilities |
| Express | Backend Web API | 5.3.0 | Routing, middleware pipeline, raw body webhook parser | Handles all /api/* endpoints and error middleware |
| MongoDB | Database | 6.0+ (Active) | NoSQL document persistence, compound indexing | Stores collections for users, rooms, bookings, payments |
| Mongoose | ODM Library | 9.11.2 | Schema validation, type casting, compound indexes | Defines 10 schemas and handles database operations |

| Technology | Category | Verified Version | Purpose in HotelPro | Connection / Interaction Point |
|---|---|---|---|---|
| JSON Web Tokens | Authentication | 9.0.3 | Stateless cryptographically signed Bearer tokens | Signed with JWT_SECRET; verified in auth.js |
| Bcrypt.js | Password Security | 3.0.3 | Adaptive one-way cryptographic password hashing | 10 salt rounds applied in authRoutes.js |
| Stripe SDK | Payment Gateway | 17.7.0 | Stripe Checkout in EUR, webhook signature verification | stripeClient.js and stripeWebhook.js |
| Supertest | Test Framework | 7.2.2 | Automated HTTP integration testing against Express | Executes 66 automated tests in api.test.js |

# Chapter 3: System Architecture

## 3.1 Multi-Tier Architecture

HotelPro follows a multi-tier decoupled architecture separating concerns across presentation, security, application, business services, persistence, and external gateways:

- Presentation Tier: React 19 SPA running on Vite 8. Communicates via Axios with automatic Bearer token injection.
- Security Perimeter: Express security middleware enforcing nosniff, DENY frame options, XSS protection, CORS origin whitelisting, and JWT signature verification.
- Application Tier: Express 5 routing layer managing 10 modular API route suites.
- Business Services: Domain services including withRoomLock Promise mutex, booking availability math, currency conversion, and tax invoice generation.
- Persistence Tier: MongoDB database managed via Mongoose 9 ODM with compound indexing, paired with an in-memory catalog of Indian heritage suites.
- External Gateway Tier: Stripe Payment Gateway processing transactions in Euros (EUR) with HMAC-SHA256 webhook signatures.

### 3.2 System Architecture Diagram

**Architecture Representation A complete Mermaid architecture diagram is provided in documentation/diagrams/architecture.mmd detailing the request flow from Client Presentation Tier through Security Perimeter, Application Routers, Business Services, External Gateways, and Data Persistence Tier.**

# Chapter 4: ER Diagram and Database Design

HotelPro defines 10 core Mongoose schemas in Backend/models/. Below is the detailed specification of every entity:

| Model Name | Primary Identifier | Key Fields and Types | References and Relationships | Purpose in HotelPro |
|---|---|---|---|---|
| User | _id (ObjectId) | name, email (UQ, indexed), password (hash), role (Enum), phone, membershipTier | Referenced by Booking, Payment, Review via email | Stores authenticated user profiles, credentials, and authorization roles. |
| Hotel | _id (ObjectId) | name, city, country, location, starRating, rating, image, images[], amenities[] | Referenced by Room.hotelId (1:N) | Represents physical resort and palace properties. |
| Room | _id (ObjectId) | roomNumber (UQ, indexed), name, type (Enum), pricePerNight, capacity, status, seasonalPricing | hotelId -> Hotel._id (N:1) | Represents bookable suites, pricing, capacity, and housekeeping status. |
| Booking | _id (ObjectId) | bookingNumber (UQ), userEmail, roomId, checkInDate, checkOutDate, nights, totalPrice, paymentStatus, holdExpiresAt, stripeSessionId | roomId -> Room._id, userEmail -> User.email | Core reservation entity tracking dates, pricing, holds, and payment statuses. |
| Payment | _id (ObjectId) | bookingId, bookingNumber, userEmail, amount, currency, paymentMethod, transactionId, status, invoiceNumber | bookingId -> Booking._id (1:1) | Financial audit record tracking transaction IDs, amounts, and tax invoices. |
| Review | _id (ObjectId) | roomId, roomName, userEmail, userName, rating (1-5), comment, staffReply | roomId -> Room._id (N:1) | Guest feedback and star ratings evaluated per suite. |

| Model Name | Primary Identifier | Key Fields and Types | References and Relationships | Purpose in HotelPro |
|---|---|---|---|---|
| Notification | _id (ObjectId) | recipientEmail, role (Enum), title, message, type, isRead | recipientEmail -> User.email | System and transaction alerts dispatched to users. |
| ServiceRequest | _id (ObjectId) | guestName, guestEmail, roomNumber, serviceType, details, priority, status, assignedStaff | guestEmail -> User.email | Concierge and housekeeping service tickets. |
| CheckInOut | _id (ObjectId) | bookingId, bookingNumber, guestName, roomNumber, checkInTime, checkOutTime, keyCardCode, status | bookingId -> Booking._id (1:1) | Front desk guest stay records with digital keycards. |
| AdminLog | _id (ObjectId) | action, performedBy (email), targetEntity, details, timestamp | performedBy -> User.email | Immutable chronological audit log of administrative actions. |

# Chapter 5: MVC Pattern and Request Lifecycle

HotelPro adapts the classic Model-View-Controller pattern for a decoupled Single Page Application (SPA) communicating with a stateless REST API:

- Model: Mongoose schemas (Backend/models/) encapsulating schema constraints, validation rules, compound indexes, and custom methods such as Booking.methods.isHoldActive().

- View: React components and modals (FrontEnd/src/) managing local component state, form validation, and responsive rendering.

- Controller: Express route handlers and controllers (Backend/routes/ and Backend/controllers/) receiving HTTP requests, coordinating business services, and formatting JSON responses.

- Services / Utilities: Domain business logic extracted into standalone modules (bookingAvailability.js, currencyConverter.js, stripeCheckout.js, emailService.js).

## 5.2 Complete Request Lifecycle Sequence

The complete reservation lifecycle proceeds through 10 deterministic steps:

- 1. Guest submits reservation form in BookingPaymentModal.jsx.

- 2. Axios client transmits POST /api/bookings with JWT Bearer token.

- 3. CORS middleware validates origin and sets security headers.

- 4. Auth middleware verifies JWT signature and attaches req.user.

- 5. Route acquires withRoomLock(roomId) Promise mutex lock.

- 6. Server queries Room.findById for authoritative rate and verifies date availability.

- 7. Server computes nights, base subtotal, and 12% statutory tax.

- 8. Currency converter converts INR total to EUR (1 EUR = 110 INR) and creates Stripe session.

- 9. Booking is persisted with paymentStatus: Pending and holdExpiresAt = now + 15m.
- 10. Mutex lock is released; client redirects browser to Stripe hosted checkout.

# Chapter 6: Features and Modules

Every feature listed below is verified against active codebase implementation:

- User Authentication: Registration, login, profile management, and 7-day JWT issuance.
- Role-Based Access Control: Strict authorization across Guest, Staff, and Admin roles.
- Suite Discovery: Multi-parameter filtering by destination, price, capacity, and amenities.
- Real-Time Availability: Half-open interval date overlap filtering excluding active bookings.
- Concurrency Mutex Lock: In-process Promise mutex per room preventing double-booking.
- 15-Minute Hold Expiry: Temporary pending holds automatically released after 15 minutes.
- Hold Retry Reuse: Pending hold reuse permitting guests to retry payment without conflict errors.
- Stripe EUR Checkout: Tariff conversion to EUR and euro cents with Stripe session creation.
- Pay at Hotel Desk: Unpaid reservation confirmation holding room permanently for desk payment.
- Dual-Path Payment Verification: Webhook HMAC verification and client redirect polling.
- Digital Tax Invoice: Printable tax invoice with line items, 12% tax, and rounding adjustments.
- Digital Keycard Check-In: Staff check-in issuing alphanumeric keycard and setting room Occupied.
- Turnover Check-Out: Staff check-out setting room Cleaning and queuing urgent service ticket.
- Housekeeping Room Grid: Real-time status management (Available, Occupied, Cleaning, Maintenance).
- Guest Service Ticketing: Service request submission, priority assignment, and fulfillment.
- Review Aggregation: 1-5 star reviews, staff replies, and room rating recalculation.
- Admin KPI Analytics: Real-time revenue aggregation, occupancy percentages, and audit logs.

# Chapter 7: Roles and Responsibilities

HotelPro strictly enforces three user roles: Guest, Staff, and Admin. Below is the verified permissions matrix:

| Operational Capability | Guest | Staff | Admin | Enforcement Point |
|---|---|---|---|---|
| Browse Rooms and Filter Availability | Yes | Yes | Yes | Public / Authenticated endpoints |
| Create Reservation and Pay via Stripe | Yes | Yes | Yes | protect middleware attached to req.user |
| Access Own Booking History and Invoices | Yes | Yes | Yes | protect with userEmail ownership check |
| Cancel Own Reservation | Yes | Yes | Yes | protect with userEmail ownership check |
| Access Other Guests Invoices / Bookings | No | Yes | Yes | Ownership check returning 403 Forbidden |
| View Staff Operational Overview Stats | No | Yes | Yes | authorize(Staff, Admin) returning 403 |

| Operational Capability | Guest | Staff | Admin | Enforcement Point |
|---|---|---|---|---|
| Check In Guest and Issue Keycard | No | Yes | Yes | authorize(Staff, Admin) |
| Check Out Guest and Queue Housekeeping | No | Yes | Yes | authorize(Staff, Admin) |
| Update Housekeeping Room Status Grid | No* | Yes | Yes | authorize(Staff, Admin, Guest)* |
| Fulfill Guest Service Requests | No | Yes | Yes | authorize(Staff, Admin) |
| View Executive KPI Analytics and Revenue | No | No | Yes | authorize(Admin) returning 403 |
| List Registered Users and Edit Roles | No | No | Yes | authorize(Admin) returning 403 |
| Inspect System Security Audit Logs | No | No | Yes | authorize(Admin) returning 403 |

# Chapter 8: Complete User Flows

HotelPro implements seven distinct end-to-end user workflows, fully modeled in Mermaid source files in documentation/diagrams/:

- Flow A: Registration and Login (bcrypt hashing, JWT issuance, forced Guest role).
- Flow B: Hotel and Suite Booking Flow (date overlap check, mutex lock, server pricing).
- Flow C: Stripe Payment in EUR Flow (canonical INR calculation, EUR cent conversion).
- Flow D: Successful Payment and Dual Verification (webhook HMAC check + redirect callback).
- Flow E: Cancelled or Failed Payment Flow (non-destructive hold, 15m retry window).
- Flow F: Booking History and Invoice Flow (auto-reconciliation, line-item tax breakdown).
- Flow G: Staff and Admin Operations Flow (keycard check-in, turnover checkout, KPI analytics).

# Chapter 9: Payment Gateway and Currency Conversion

This chapter details the mathematical and operational specifications of HotelPro Stripe EUR payment gateway integration.

## 9.1 Mathematical Conversion Specification

The currency conversion pipeline bridges domestic Indian Rupee tariffs with international credit card processing in Euros (EUR):

- USD Base Rate to Canonical INR: The server converts the base room rate to INR using the configured rate ($1 = 83 INR via USD_TO_INR_RATE).
- INR to EUR Conversion: The canonical INR tariff is converted to Euros using the configured rate (1 EUR = 110 INR via EUR_TO_INR_RATE).

- Euro Cents Calculation: The EUR amount is multiplied by 100 to produce an exact integer for Stripe unit_amount.

### 9.2 Worked Mathematical Example

**Illustrative Numerical Walkthrough Suite: Pink City Deluxe Heritage Chamber**

**Base Rate: .00 / night | Duration: 2 nights**

**Base Subtotal: .00 x 2 = .00**

**12% Statutory Tax: .00 x 0.12 = .60**

**Total USD Tariff: .00 + .60 = .60**

**Canonical INR Total: Math.round(.60 x 83) = INR 44,621**

**EUR Conversion: 44,621 / 110 = 405.64545... -> EUR 405.65**

**Euro Cents for Stripe: 405.65 x 100 = 40,565 cents**

**Line-Item Reconciliation on Invoice:**

**Unrounded Base: 405.65 / 1.12 = 362.1875 EUR**

**Rate per Night: EUR 181.09 / night**

**Subtotal: EUR 181.09 x 2 = EUR 362.18**

**12% Occupancy Tax: Math.round(362.18 x 0.12) = EUR 43.46**

**Subtotal + Tax: EUR 362.18 + EUR 43.46 = EUR 405.64**

**Rounding Adjustment: EUR 405.65 - EUR 405.64 = EUR 0.01**

**Reconciled Invoice Total: EUR 362.18 + EUR 43.46 + EUR 0.01 = EUR 405.65 (Exact Match)**

# Chapter 10: API Documentation

HotelPro exposes 46 REST endpoints organized across 10 functional modules. Below is the API summary:

| Module | Method | Endpoint Path | Auth Required | Allowed Roles | Purpose |
|---|---|---|---|---|---|
| Auth | POST | /api/auth/register | No | Public | Register guest account (role forced to Guest) |
| Auth | POST | /api/auth/login | No | Public | Authenticate credentials and issue 7-day JWT |
| Auth | GET | /api/auth/me | Yes | Any | Retrieve authenticated user profile |
| Hotels | GET | /api/hotels | Yes | Any | List hotel properties with room counts |
| Rooms | GET | /api/rooms | No | Public | Search rooms with date-range availability filter |
| Rooms | POST | /api/rooms | Yes | Admin | Create new luxury room |
| Bookings | POST | /api/bookings | Yes | Any | Create reservation with mutex lock and hold |
| Bookings | GET | /api/bookings/my-bookings | Yes | Any | List bookings for authenticated guest |
| Bookings | POST | /api/bookings/:id/cancel | Yes | Owner/Staff | Cancel booking and trigger Stripe refund |
| Payments | POST | /api/payments/checkout | Yes | Owner/Staff | Create Stripe Checkout session in EUR |
| Payments | POST | /api/payments/stripe/verify | Yes | Owner/Staff | Verify Stripe session and mark Paid |
| Payments | POST | /api/payments/stripe-webhook | No | Stripe Signature | Handle Stripe webhook events (HMAC verified) |
| Payments | GET | /api/payments/invoice/:id | Yes | Owner/Staff | Generate reconciled digital tax invoice |
| Staff | GET | /api/staff/stats | Yes | Staff, Admin | Retrieve operational dashboard statistics |

| Module | Method | Endpoint Path | Auth Required | Allowed Roles | Purpose |
|---|---|---|---|---|---|
| Staff | POST | /api/staff/checkin | Yes | Staff, Admin | Check in guest and issue digital keycard |
| Staff | POST | /api/staff/checkout | Yes | Staff, Admin | Check out guest and queue housekeeping ticket |
| Staff | PUT | /api/staff/rooms/:id/status | Yes | Staff/Admin/Guest | Update housekeeping room status grid |
| Admin | GET | /api/admin/analytics | Yes | Admin | Retrieve executive revenue and occupancy KPIs |
| Admin | GET | /api/admin/users | Yes | Admin | List registered user directory |
| Admin | PUT | /api/admin/users/:id/role | Yes | Admin | Update user authorization role or tier |

# Chapter 11: Security and Validation

HotelPro implements multi-layered security controls:

- Password Hashing: 10 salt rounds of adaptive Blowfish hashing via bcryptjs.
- Role Escalation Prevention: Client-supplied roles in registration payloads are unconditionally ignored.
- Server-Authoritative Pricing: Nightly rates, nights count, and 12% statutory tax calculated from MongoDB.
- Double-Booking Mutex Lock: withRoomLock Promise mutex serializes concurrent checkouts per room.
- Cryptographic Webhook Verification: Stripe webhooks validated via constructEvent using raw body.
- IDOR Protection: Ownership validation enforces that guests only access their own bookings and invoices.
- HTTP Security Headers: nosniff, DENY frame options, XSS protection, strict referrer policy.
- CORS Whitelist: Requests validated against approved origins (localhost:5173, localhost:3000).

# Chapter 12: Testing and Verification

The automated test suite in Backend/test/api.test.js executes 66 end-to-end integration tests using Supertest:

- Phase 1: Authentication and Security Gate (Tests 1-8: registration, bcrypt, duplicate rejection).
- Phase 2: Role-Based Access Control (Tests 9-12: 403 Forbidden enforcement on admin/staff routes).
- Phase 3: Booking Logic and Availability (Tests 13-18: server pricing, overlap rejection, cancellations).

- Phase 4: Notifications and Reviews (Tests 19-20: recipient isolation, star rating validation).
- Phase 5: Route Protection and Search (Tests 21-24: date filtering, my-bookings endpoint).
- Phase 6: Payment Integrity and CORS (Tests 25-29: desk booking, simulated card rejection, CORS policy).
- Phase 7: Payment Verification (Tests 30-36: signature validation, tampered signature rejection).
- Phase 8: Complete Guest Journey (Tests 37-40: register -> login -> search -> book -> invoice).
- Phase 9: Destination Filtering and Regression (Tests 41-51: destination searches, suite bookings).
- Phase 10: Hold Expiry and Retries (Tests 52-60: 15-minute hold, hold reuse, expired hold release).
- Phase 11: Stripe EUR Conversion (Tests 61-66: rate check, cent rounding, invoice reconciliation).

# Chapter 13: Installation and Execution

HotelPro requires Node.js >= 18 and MongoDB >= 6.0:

- Backend Setup: Navigate to Backend/, run npm install, configure .env, and start with npm run dev.
- Frontend Setup: Navigate to FrontEnd/, run npm install, configure .env, and start with npm run dev.
- Automated Seeding: If the database contains fewer than 4 hotels or 8 rooms, the backend automatically seeds luxury hotels, heritage suites, demo users, and bookings on startup.
- Demo Evaluation Accounts: Guest (guest@hotelpro.com), Staff (staff@hotelpro.com), Admin (admin@hotelpro.com) - password: password123.

# Chapter 14: Limitations and Future Enhancements

Known limitations identified through inspection:

- Stripe Webhooks in Local Dev: Webhooks require a public tunnel (e.g. Stripe CLI) for asynchronous delivery; mitigated via synchronous redirect verification.
- Static Exchange Rates: EUR (1 EUR = 110 INR) and USD ($1 = 83 INR) rates are statically configured in .env rather than fetched from live forex APIs.
- Single-Process Mutex: withRoomLock is an in-process lock; horizontal scaling across multiple servers would require distributed locking such as Redis Redlock.
- Future Roadmap: Dynamic forex API feeds, Redis distributed locking, IoT smart lock integration, and loyalty points rewards engine.

# Chapter 15: Conclusion

HotelPro represents an enterprise-grade hospitality management system. By combining an aesthetically rich React 19 frontend with a robust, security-hardened Express 5 backend, HotelPro delivers seamless guest booking, deterministic concurrency control, transparent international payment processing in Euros, and real-time operational workflows for staff and administrators.

With 66 automated integration tests validating every functional and security boundary, the platform stands fully verified and ready for formal evaluation and deployment.

# Appendices

## Appendix A: Repository File Structure

HotelPro-Capstone contains Backend/ (controllers, middleware, models, routes, test, utils), FrontEnd/ (src/api, src/components, src/context, src/pages), and documentation/ (diagrams, technical documentation).

## Appendix B: Glossary of Technical Terms

- JWT: JSON Web Token used for stateless Bearer authentication.
- RBAC: Role-Based Access Control enforcing permissions across Guest, Staff, and Admin.
- Mutex: Mutual exclusion lock serializing concurrent operations per room ID.
- holdExpiresAt: 15-minute expiration timestamp releasing abandoned online reservations.
- Euro Cents: Smallest integer currency unit processed by Stripe (EUR amount x 100).
- Rounding Adjustment: Line-item entry reconciling sub-cent fractions on digital tax invoices.